

7.4. Рекурсия и поисковые задачи

Рекурсия и комбинаторный перебор. Самым очевидным и неинтеллектуальным способом решения поисковых задач является полный перебор всех возможных вариантов. Когда речь идет о переборе множества последовательных значений, множества возможных пар и т.п., то программа содержит один или несколько вложенных циклов (см. 2.3.). Когда же речь идет о переборе возможных сочетаний, подмножеств, то есть о комбинаторике, то рекурсия возникает естественным образом. Очередной шаг алгоритма рассматривает возможные варианты увеличения последовательности (подмножества) на один элемент и таким образом получает из одной задачи размерности n несколько задач размерности $n+1$, для которых он естественным образом рекурсивно повторяется.

$$\sum_{i=0}^n \alpha_i$$

Комбинаторный генератор вариантов. Сначала попробуем смоделировать в программе процесс построения требуемой последовательности (подмножества) значений из исходного множества. Парадокс рекурсивного подхода состоит в том, что перед началом разработки следует предположить, что задача уже частично решена (на самом деле – предыдущими шагами рекурсии), то есть из n элементов исходного множества R построена требуемая последовательность W размерности k . Рекурсивная функция должна рассмотреть все возможные варианты «добавления» к W еще одного элемента из R , для каждого из этих вариантов получается аналогичная задача размерности $n+1$, которая решается рекурсивным вызовом той же функции. Завершение рекурсии состоит в получении очередного «окончательного» варианта, когда текущая размерность задачи n достигает размерности исходного множества R .

Вид последовательности	Алгоритм n -го шага	Ветвление	T
Все элементы	Цикл for ($i=0; i < n; i++$)	---	n
Все пары	Цикл в цикле for ($i=0; i < n-1; i++$) for ($j=i-1; j < n; j++$)	---	$n(n-1)/2$
Все подмножества	Очередной (n -ый) элемент из R может быть «добавлен» и «не добавлен» в W	2	2^n
Подмножества из n по m	Очередной (n -ый) элемент из R может быть «добавлен» и «не добавлен» в W при ограничении их количества в W	$2(1,0)$	$\frac{N!}{m!(n-m!)}$
Последовательность без повторений	В W может быть добавлен любой из R , который в нем остался	$n \dots 1$	$N!$
Последовательность с повторениями	В W может быть добавлен любой из R	n	n^n

Первые два варианта моделируют процесс, в котором на каждом шаге рассматривается две альтернативы для очередного элемента исходного множества: включать и не включать в выходное, соответственно имеют место два рекурсивных вызова для следующей ($n+1$) размерности. Окончание рекурсии в первом случае происходит по завершении перебора исходного множества, во втором – при накоплении заданного количества элементов.

```

//-----74-01
// Множество всех подмножеств
// R - исходное множество
// W - результирующее множество (последовательность)
// n - номер шага в глубину - индекс выбираемого элементов из R
// k - количество выбранных элементов
// N - размерность задачи
// cnt - счетчик вариантов
void F1(int W[],int R[],int n,int k,int N, int &cnt){
int i;
if (n==N){
    cnt++; printf("\n");
    for (i=0;i<k;i++) printf("%d ",W[i]);
    return; }
    // достигнута требуемая размерность
    // подсчет и вывод полученного варианта
    // очередной не включается
F1(W,R,n+1,k,N,cnt);
W[k]=R[n];
F1(W,R,n+1,k+1,N,cnt);
    // очередной включается
}
//-----
// Сочетаний из n по m
// m - размерность подмножества
void F2(int W[],int R[],int n,int k,int N, int m, int &cnt){
int i;
if (n>N) return;
if (k==m){
    cnt++; printf("\n");
    for (i=0;i<k;i++) printf("%d ",W[i]);
    return; }
    // превышена размерность множества
    // выбрано m из n
    // подсчет и вывод полученного варианта
    // очередной не включается
F2(W,R,n+1,k,N,m,cnt);
W[k]=R[n];
F2(W,R,n+1,k+1,N,m,cnt);
    // очередной включается
}

```

Для поисковых задач, в которых просматриваются все возможные последовательности (цепочки, слова и т.п., возможно с дополнительными ограничениями) имеется другой способ перебора. В нем на каждом шаге в последовательность переносится один из оставшихся элементов, причем перед следующим шагом он удаляется из исходного множества, а по возвращении – восстанавливается.

```

//-----
// Последовательность без повторений
void F3(int W[],int R[],int n, int N, int &cnt){
int i;
if (n==N){
    cnt++; printf("\n");
    for (i=0;i<n;i++) printf("%d ",W[i]);
    return; }
for (i=0;i<N;i++){
    if (R[i]==0) continue;
    W[n]=R[i];
    R[i]=0;
    F3(W,R,n+1,N,cnt);
    R[i]=W[n];
}
}
    // цикл по всем оставшимся
    // пропуск уже выбранного
    // выбор оставшегося
    // исключение из исходного
    // возвращение выбранного

```

Более элегантный вариант просто меняет местами очередной элемент со всеми оставшимися.

```

//-----
// Последовательность с повторениями - обмен с оставшимися
void F4(int R[],int n,int N, int &cnt){
int i,c;
if (n==N){

```

```

        cnt++; printf("\n");
        for (i=0;i<n;i++) printf("%d ",R[i]);
        return; }
F4(R,n+1,N,cnt); // очередной на своем месте
for (i=n+1;i<N;i++){
    c=R[n]; R[n]=R[i]; R[i]=c; // обмен очередного с оставшимися
    F4(R,n+1,N,cnt);
    c=R[n]; R[n]=R[i]; R[i]=c; // возвращение очередного
}}

```



Линейный кроссворд. Для заданного набора слов требуется построить линейный кроссворд. Если окончание одного слова совпадает с началом следующего более чем в одной букве (например, МАТРАС-РАСИСТ), то такие слова можно объединить в цепочку. Первоначально ставится задача - получить любую такую цепочку, окончательно - цепочку минимальной длины.

Решение задачи базируется на комбинаторном переборе всех последовательностей без повторений, рекурсивная функция для генерации которых была рассмотрена выше. В сущности, новая программа представляет собой переложение упомянутой на другую структуру данных, тем не менее проведем содержательное проектирование такой программы «с нуля» для иллюстрации технологии. Начало проектирования любой рекурсивной программы заключается в определении шага рекурсивного процесса. Пусть имеется уже составленная цепочка из выбранных слов. Очередной шаг процесса состоит в попытке присоединения к имеющейся цепочке еще одного слова из оставшихся. Если это возможно, то для новой цепочки необходимо выполнить попытку присоединить следующее слово и так далее, то есть выполнить следующий шаг рекурсивного процесса. Таким образом:

- рекурсивная функция выполняет попытку присоединения очередного слова к уже выстроенной цепочке;
- результатом функции является логическое значение (данную цепочку можно достроить), функция ищет первый подходящий вариант;
- условием завершения рекурсии является отсутствие еще не присоединенных к цепочке слов (успешное завершение), либо невозможность завершения цепочки ни через одно из оставшихся слов (неудача).

Множество исходных слов является глобальной структурой данных - массивом указателей, из которого на каждом шаге извлекается очередной элемент, но по завершении просмотра варианта (после рекурсивного вызова) возвращается обратно. Будем использовать массив указателей на строки. Извлечение строки из множества будет заключаться в установке указателя на строку нулевой длины.

И еще несколько мелочей, которые дополняют общую картину:

- проверка условия завершения рекурсии – достаточно подсчитывать число шагов рекурсии – при успешном завершении оно равно размерности структуры данных;
- сама цепочка выбранных слов может формироваться как в виде прямого, так и в виде обратного накопления результата. В данном примере слова просто выводятся на экран в обратном порядке в цепочке возвратов при достижении положительного решения «положительных» возвратов. Для большей образности функция подсчитывает количество символов, накопленных в цепочке, для вывода соответствующего количества пробелов перед словом.

```

//-----74-02.cpp
//---- Линейный кроссворд

```

```

char *w[]={"sinus","prefix","setup","plintus","drop",NULL};
int N=sizeof(w)/sizeof(char*)-1;
// lw - текущее слово цепочки, s - число символов в цепочке
int step(char *lw, int s, int k){
    if (k==N) return 1; // Все слова выбраны - успех
    for (int n=0; w[n]!=NULL;n++){
        { int l=strlen(lw); // проверка на присоединение
          if (w[n][0]==0) continue; // слово уже выбрано - пропустить
          if (k==0 || lw[l-1]==w[n][0]){ // совпадение символов или первое слово
              char *pw=w[n]; // запомнить и исключить
              w[n]=""; // проверяемое слово из множества
              if (step(pw,s+l-1,k+1)){ // попытка вывести цепочку из нового слова
                  for (int i=0;i<s+l-1;i++) // удача - вывести слово и выйти
                      putchar(' '); // из текущего шага рекурсии
                  puts(pw);
              }
              return 1;
          }
          w[n]=pw; // вернуть исключенное слово
        }
    }
    return 0; }
void main() { step("",1,0); }

```

Второй вариант генерации возможных последовательностей (см. выше) просто меняет местами очередной элемент со всеми оставшимися при соответствующем совпадении символов:

- для очередного (**i**) слова просматриваются все последующие;
- если начало одного из них (**j**) совпадает с окончанием текущего, то оно переставляется со следующим (**i+1**), то есть замещает в возможной цепочке **i+1** слово;
- если рекурсивный вызов не смог достроить цепочку, то переставленные слова возвращаются на свои места;
- при успешном завершении слова будут расположены в нужном порядке.

Кроме того, этот пример использует способ выхода из цепочки рекурсивных вызовов через исключение (см. 12.4).

```

//-----74-03.cpp
//---- Линейный кроссворд (перебор с обменом строк)
char *w[]={"sinus","prefix","setup","plintus","drop",NULL};
// k - номер шага
void step(int k){
    if (w[k+1]==NULL) throw 1; // Все слова выбраны - успех
    int l=0; if (k!=-1) l=strlen(w[k])-1; // Выход из цепочки рекурсивных вызовов
    for (int n=k+1; w[n]!=NULL;n++) // через исключение
        if (k==-1 || w[k][l]==w[n][0]){ // совпадение символов или первое слово
            char *q=w[k+1]; w[k+1]=w[n]; w[n]=q; // Обмен найденного со следующим за...
            step(k+1); // попытка вывести цепочку из нового слова
            q=w[k+1]; w[k+1]=w[n]; w[n]=q; // Обратный обмен для продолжения поиска
        }
}
void main(){
    try { step(-1); }
    catch(int v){ // Обработчик исключения -
        for (int s=0,i=0; w[i]!=NULL; i++){ // вывод слов «лесенкой» с подсчетом
            for(int j=0;j<s;j++) putchar(' '); // числа символов в предыдущих словах
            s+=strlen(w[i])-1;
            puts (w[i]);
        }
    }
}

```



Полный обход графа. Задача коммивояжера. Требуется найти минимальный путь обхода всего множества «городов», не посещая один и тот же город дважды (задача коммивояжера). Эта задача решается аналогично на основе схемы полного перебора возможных последовательностей, однако способ реализации такого перебора несколько отличается от предыдущего. Технология проектирования рекурсивных алгоритмов дает здесь следующий набор решений:

1. Представление графа – матрица расстояний между соседними городами (**R**) (матрица смежности), нулевое значение **R_{ij}** – отсутствие прямого пути.
2. Шаг рекурсивного алгоритма – при «посещении» очередного **k**-го города делается попытка движения ко всем возможным соседям, для которых решается аналогичная задача.
3. Используется прямое накопление результата – длины пройденного пути - **Int** и последовательности обхода - **D**, текущее значение минимального пути и оптимальной последовательности обхода (**Intmin** , **Dmin**) представляют собой глобальные данные и корректируются при обнаружении очередного решения («успешного» завершения рекурсии).
4. Начальное состояние шага рекурсивного алгоритма – формальные параметры рекурсивной функции: номер текущего «города» (**k**), количество пройденных городов (шагов рекурсии- **n**), длина пройденного пути (**Int**).
5. Ограничения рекурсии – обнаружение «успеха»: пройдены все города (**n** равно размерности задачи), отсечение «неудач»: повторное посещение одного и того же города;
6. Последовательность обхода городов – глобальные данные рекурсивного алгоритма, каждый шаг рекурсии добавляет в нее очередной элемент – **D[n]=k**. Но для более эффективной проверки факта повторного «посещения» используется массив «отметок» пройденных городов, индексируемый уже не по номеру шага алгоритма, а по номеру «города». Рекурсивный алгоритм в начале очередного шага проверяет (**M[k]==0**) и устанавливает (**M[k]=1**) значение этого признака, а по окончании - сбрасывает его.
7. Правила перебора возможных вариантов: цикл по строке матрицы смежности – перебор возможных соседей с пропуском тех, до которых нет прямого пути (**R[k][i]=0**);
8. Начальное состояние следующего шага – фактические параметры рекурсивного вызова: «номер соседнего города» - индекс цикла перебора (**i**), количество шагов рекурсии – **n+1**, длина пройденного пути – **Int+R[k][i]**;
9. Условия первоначального вызова рекурсивной функции в main – **step(0,0,0)**.

```
//-----74-04.cpp
//---- Задача коммивояжера - полный обход графа
#define N 5
int R[N][N]={0,4,2,0,0},{4,0,2,1,3},{2,2,0,0,6},{0,1,0,0,1},{0,3,6,1,0};
int M[N]={0,0,0,0,0}; // Отметка пройденных "городов"
int W[N]; // Текущая последовательность обхода
int Wmin[N]; // Оптимальная последовательность обхода
int minInt=-1; // Длина минимального пути
void step(int n, int k, int Int){ // n - номер шага, k - номер "города"
if (n==N){ // Int - длина пройденного пути
    if (minInt== -1 || Int<minInt) // Обход закончен - фиксировать минимум
    {
        minInt=Int; // Запомнить длину и последовательность
        for (int i=0;i<N;i++) // обхода
            Wmin[i]=W[i];
    }
}
```

```

    }
    return;
}
if (M[k]==1) return; // Повторное прохождение
W[n]=k; // Дополнить последовательность обхода
M[k]=1; // Отметить прохождение
for (int i=0; i<N; i++){ // Просмотр соседей
    if (R[k][i]==0) continue; // Соседи не связаны - пропустить
    step(n+1,i,Int+R[k][i]); // Рекурсивный вызов для соседнего
} // "города" с учетом расстояния до него
M[k]=0; } // Сбросить отметку

void main(){ step(0,0,0); printf("\nmin=%d\ntowns:",minInt);
for (int i=0; i<N; i++) printf("%d-",Wmin[i]); }

```

Головоломка с переливанием. Требуется при помощи двух емкостей 3 и 5 литров получить во второй емкость 4 литра. Рекурсивный характер решения задачи методом полного перебора возможных вариантов состоит в том, что для каждого варианта «разлива» (заполнения емкостей) имеется 6 смежных вариантов (аналогичных подзадач), получаемых из исходного:

- выливанием жидкости из одной из емкостей;
- доливанием жидкости в одну из емкостей (до края);
- переливанием жидкости из одной емкости в другую с учетом того, сколько жидкости осталось в исходной емкости и сколько места есть в другой (из этих двух значений нужно выбрать минимум).

Единственная особенность решения задачи состоит в том, что промежуточные варианты разлива нужно запоминать, чтобы избежания заикливания рекурсии. Программа использует полный комбинаторный перебор возможных вариантов с «прямым» накоплением результата и запоминанием оптимального. Заведомо неэффективные варианты отсекаются при обнаружении того факта, что текущая последовательность операций становится длиннее уже имеющейся оптимальной.

```

//-----74-05.cpp
// Головоломка: с помощью банок 3 и 5 литров получить 4 литра
#define P1 3 // Емкости банок
#define P2 5
#define EX 4 // Искомый результат
#define N 1000

struct var{
    int nmax; // Количество операций
    int V[N][2]; // Вариант "разлива"
    int D[N]; // Следующее действие
} VAR,OPT={-1};

int cnt=0; // Счетчик рекурсивных вызовов

void F(int v1,int v2, int n){ // v1,v2 - текущее заполнение "банок"
    VAR.V[n][0]=v1; // Сохранение нового варианта "разлива"
    VAR.V[n][1]=v2;
    VAR.D[n]=0;
    if (v2==EX) {
        if (OPT.nmax==-1 || n < OPT.nmax)
            OPT=VAR, OPT.nmax=n;
        return; } // Найдено требуемое заполнение
    if (OPT.nmax!=-1 && n>=OPT.nmax)
        return; // Есть уже более оптимальное решение
    cnt++;
    for (int i=0;i<n;i++) // Проверка на повторение "разлива"

```

```

    if (VAR.V[i][0]==v1 && VAR.V[i][1]==v2) return;
// Возможные подзадачи - заполнение и опустошение каждой "банки"
VAR.D[n]++; F(P1,v2,n+1);
VAR.D[n]++; F(v1,P2,n+1);
VAR.D[n]++; F(0,v2,n+1);
VAR.D[n]++; F(v1,0,n+1);
// Возможные подзадачи - переливание из одной "банки" в другую
// Объем переливаемой жидкости -
// Минимум из остатка в первой и свободного места во второй
int dv=(v1 < P2-v2 ? v1 : P2-v2);
VAR.D[n]++; if (dv!=0) F(v1-dv, v2+dv,n+1);
dv=(P1-v1 < v2 ? P1-v1 : v2);
VAR.D[n]++; if (dv!=0) F(v1+dv, v2-dv,n+1); }

```

Лабораторный практикум

1. Тройки и семерки. Какое наименьшее число обладает тем свойством, что оно записывается только с помощью цифр 3 и 7 и что как оно, так и сумма его цифр делятся на 3 и 7? Например, 7 733 733 делится без остатка на 3 и на 7, но сумма его цифр (33) на 3 делится, а на 7 нет, поэтому оно не может служить решением задачи.
2. Задача о десяти цифрах. Расставьте все десять цифр 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 в таком порядке, чтобы получившееся число делилось на все числа от 2 до 18. Если, например, разместить цифры в последовательности 1 274 953 680, то получившееся число будет делиться на 2, 3, 4, 5 и т. д. до 16, но не разделится на 17.
3. Между цифрами от 1 до 9 расставить знаки операций +,-,*,/ так, чтобы получилось заданное число.
4. Между цифрами от 1 до 9 расставить знаки операций +,- так, чтобы получилось заданное число, из подряд идущих цифр можно составлять многозначные, например 123-45+7+8-9.
5. Реализовать рекурсивный алгоритм построения цепочки из имеющегося набора костей домино.
6. На шахматной доске имеется N фигур противника. Разместить на ней минимальное количество коней так, чтобы все фигуры противника находились «под боем».
7. На шахматной доске имеется N фигур противника. Разместить на ней минимальное количество слонов так, чтобы все фигуры противника находились «под боем».
8. Имеется 2 строки символов А и В, например, АААВАВ и АВААВВВ. Алгоритм просматривает пару строк и имеет право выполнить одну из команд (удалить очередной символ первой (второй) строки, вставить символ А(В) в текущую позицию первой (второй) строки. Определить минимальную последовательность команд для преобразования строк.
9. Задан словарь и слов одинаковой длины, например «барак, баран, банан, дурак, бутон, бутан, питон, барин». Для заданной пары слов определить цепочку так, чтобы каждое следующее слово отличалось не более чем на 2 буквы.
10. Программа генерирует текст из строки, содержащей определения циклических фрагментов вида ...(Иван,Петр,Федор=Жил-был * (синего,белого,желтого= у самого * моря)).... Символ «*» определяет место подстановки имени из списка в очередное повторение фрагмента. Допускается вложенность фрагментов. Полученный текст помещается в выходную строку.

11. Задан набор слов (массив указателей на строки). Построить из них любую цепочку таким образом, чтобы символ в начале следующего совпадал с одним из символов в середине предыдущего (не первым и не последним). При выводе разместить слова с отступом от начала строки так, чтобы они соответствовали совпадению букв.
12. Задача раскраски карты. Страны на карте заданы матрицей смежности. Если страны i, j имеют на карте общую границу, то элемент матрицы $A[i, j]$ равен 1, иначе 0. Смежные страны не должны иметь одинакового цвета. «Раскрасить» карту минимальным количеством цветов (генерация матрицы смежности в `programms/randommatrix.cpp`).
13. Задача о разноцветных проводах. Имеется система узлов и соединений. Если узлы i, j соединены между собой, то элемент матрицы $A[i, j]$ равен 1, иначе 0. Определить минимальное количество проводов разного цвета для выполнения соединений так, чтобы в каждом узле не было двух одинаковых цветов (генерация матрицы смежности в `programms/randommatrix.cpp`).
14. Задача проведения границы на карте («создание военных блоков»). Страны на карте заданы матрицей смежности. Если страны i, j имеют на карте общую границу, то элемент матрицы $A[i, j]$ равен 1, иначе 0. Необходимо разбить страны на две группы так, чтобы количество пар смежных стран из противоположных групп было минимальным (генерация матрицы смежности в `programms/randommatrix.cpp`).

Вопросы без ответов

Содержательно сформулируйте результат выполнения рекурсивной функции, укажите в тесте технологические приемы рекурсивного программирования. Оцените трудоемкость рекурсивного алгоритма (аналитически). Добавьте в функцию счетчик рекурсивных вызовов (или базовых операций – сравнения, перемещения, обмена). Сравните теоретическое и экспериментальное значения трудоемкости для различных размерностей задачи.

Пример выполнения тестового задания.

```
//-----
void FF(int A[], int n){
    if (n==1) return;
    for (int j=1, k=0; j<n; j++){
        if (A[j]>A[k]) k=j;
    }
    int v=A[k]; A[k]=A[n-1]; A[n-1]=v;
    FF(A, n-1);
}
```

Основу теста составляет линейная рекурсия: размерность задачи на каждом шаге уменьшается на 1. Функция находит максимальный элемент массива и меняет его с последним, после чего рекурсивно вызывает себя для части массива, не включающей последний элемент. В результате имеет место сортировка выбором. Рекуррентная формула трудоемкости для шага рекурсии - $T_N = T_{N-1} + N$, для которой общая трудоемкость - $T = N^2/2$, учитывая, что количество сравнений - $N-1$, и $T_N = T_{N-1} + N-1$, получим $T = N(N-1)/2$.

int FF(int A[], int n){	// Трудоемкость m – количество сравнений
if (n==1) return 0;	// Ограничение рекурсии
for (int m=0, j=1, k=0; j<n; j++, m++){	// Найти максимальный в массиве
if (A[j]>A[k]) k=j;	
int v=A[k]; A[k]=A[n-1]; A[n-1]=v;	// Обменять с последним
return FF(A, n-1) + m; }	// Рекурсивный вызов для размерности, меньшей на 1


```

void main() { int v=0,B[10]={4,8,3,4,8,2,3,8,3,6};
printf("T=%d\n",FF(B,10));
for (int i=0;i<10;i++) printf("%d ",B[i]);
}

//-----74-06.cpp
//----- 1
int F1(int n)
{ if (n==1) return 1;
return (n * F1(n-1)); }
//----- 2
int F2(int A[],int a, int b)
{ if (a>b) return 0; if (a==b) return A[a];
int m=(a+b)/2;
int v1=F2(A,a,m),v2=F2(A,m+1,b);
return v1>v2 ? v1: v2; }
//----- 3
int F3(int A[],int a, int b)
{ if (a>b) return 0; if (a==b) return A[a];
int m=(a+b)/2;
return F3(A,a,m)+F3(A,m+1,b); }
//----- 4
void F4(int A[], int a, int b)
{ int i,j,mode;
if (a>=b) return;
for (i=a, j=b, mode=1; i != j; mode >0 ? i++ : j--)
if (A[i] > A[j])
{ int c;
c=A[i]; A[i]=A[j]; A[j]=c; mode = -mode;
}
F4(A,a,i-1); F4(A,i+1,b); }
//----- 5
int F5(int A[], int a, int b, int val)
{ int i,j,mode;
if (a>=b) return -1;
int m=(a+b)/2;
if (val==A[m]) return m;
if (val<A[m]) return F5(A,a,m-1,val);
return F5(A,m+1,b,val); }
//----- 6
long F6(int n){
if (n==0 || n==1) return 1;
return F6(n-1)+F6(n-2);}
//----- 7
char *F7(char *p, char *s) {
if ( *s =='\0') return p;
*p++ = *s;
p=F7(p, s+1);
*p++ = *s;
return p;}
void z7() { char *q, S[80]; *F7(S, "abcd")=0; }
//----- 8
void F8(char *&p, char *s){
if ( *s =='\0') return;
*p++ = *s;
F8(p, s+1);
*p++ = *s; }
void z8() {char *q, S[80]; q = S; F8(q,"abcd"); *q=0; }
//----- 9
double F9(double *pk, double x, int n)
{ if (n==0) return(*pk);
return *pk + x *F9(pk+1,x,n-1); }
void z9() {double B[]={5.,0.7,4.,3.},X=3.,Y=F9(B,X,4); }
//----- 10

```

```
void F10(int *p, int nn)
{ if (nn==1) { *p=0; return; }
  for (int i=2; nn % i !=0; i++);
  *p=i; F10(p+1,nn/i); }
```